.OpenHarmony Compile Agent使用指南（opendesk）

目录

1. 项目概述
2. 环境要求
3. 安装部署 — 14 步设置向导详解
4. 通过 AI Agent完成贡献（推荐）
5. 以cunit命令行工具为例梳理整个过程（重点）

1. 项目概述

1.1 是什么

OpenHarmony Compile Agent 是面向 OpenHarmony（鸿蒙）aarch64 平台的 **AI 驱动的自动化开源软件编译框架**，由 OpenHarmony PC Developer 社区维护（🔗 AtomGit - 全球开发者的开源社区,开源代码托管平台）。它的核心使命是：把常见 Linux 开源软件在鸿蒙平台上完成 **源码→编译→打包→归档** 的全流程自动化，并将每一次踩坑经验沉淀为可机器匹配的知识图谱。

1.2 解决什么问题

在鸿蒙 PC 开发者环境中，大量 Linux 开源软件需要针对 **aarch64 + musl + 平台特有约束**（如 `/tmp` 只读、HMDFS 文件系统、SELinux 安全策略、ELF 签名要求、`gcc` 不存在仅有 `clang` 等）完成编译适配。传统手工方式下，每个库一套脚本、每台机器一点差异，排障靠口口相传。本框架将这些问题系统化解决。

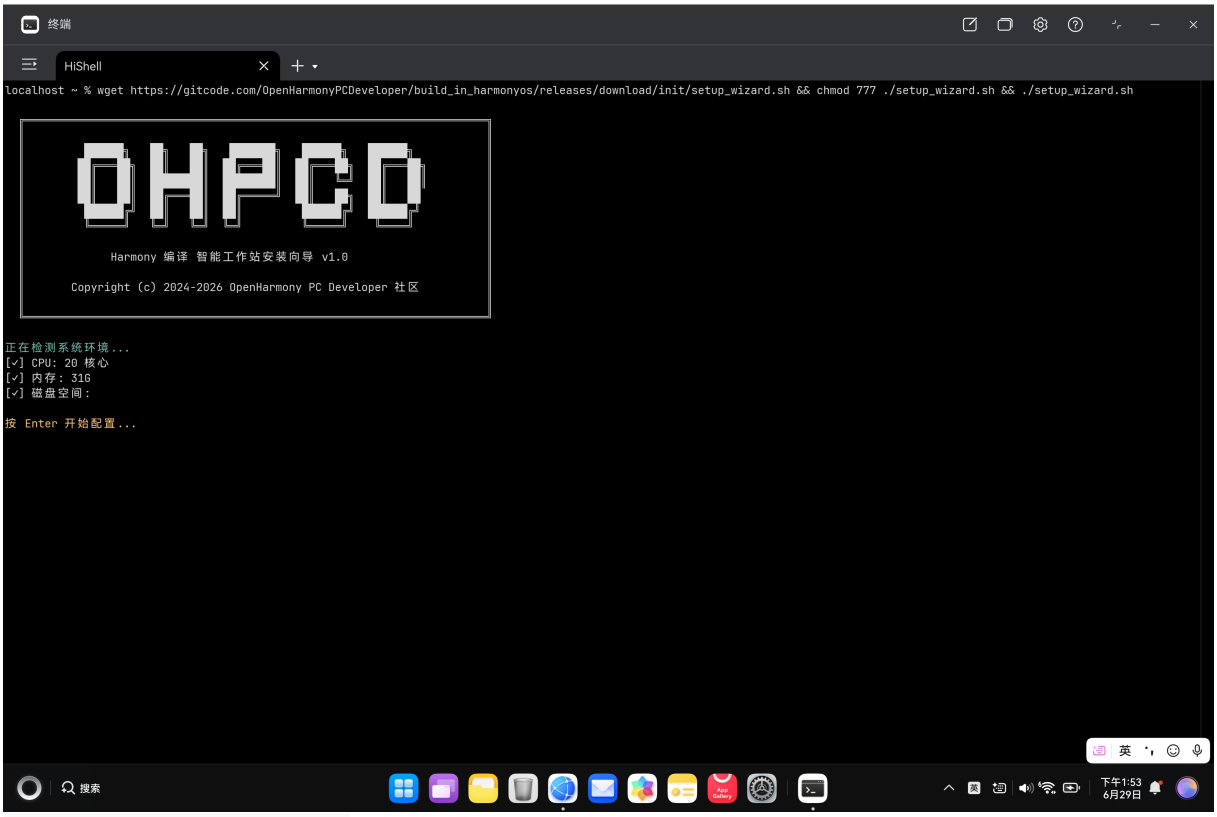
1.3 核心设计理念

项目遵循六大设计原则：

原则	说明
零 bash 依赖	所有逻辑纯 Python 实现（29 个模块），无 shell 脚本陷阱
知识自增长	每编译一个软件，知识图谱就多一条经验
边做边记	编译过程中实时记录问题和方案，避免编译完成后遗忘
预编译包优先	缺依赖时优先安装预编译包，避免编译链过长
跨对话恢复	编译中断后可通过 `ohc-run.py status` 查看状态并从断点恢复
DESTDIR 路径隔离	配置、打包、部署三个阶段路径分离，互不污染

核心设计公式：

AI (逻辑推理) + 结构化约束 (SKILL/铁律) + 经验闭环 (错误模式匹配) =



会成长的自动化工具

1.4 项目当前规模

指标	数值
已归档软件	**394 个**
已发布版本	**309 个**
知识图谱错误模式	**99 条**（E001-E104）
预扫描规则	**15 条**
运维文档	**13 篇**
CLI 子命令	**25 个**
Python 模块	**29 个**（零外部依赖）
质量门禁	**5 道**（G0-G4）
铁律	**20 条**（R1-R25）

1.5 双仓库架构

仓库	定位
`build_in_harmonyos`	编译框架 + 编译档案（SKILL、补丁、笔记、报告）
`cmd-pkgs`	预编译 `.tar.gz` 分发侧，提供一键安装脚本

2. 环境要求

2.1 硬件要求

- **架构**：aarch64（ARM 64 位）
- **运行平台**：鸿蒙 PC（HarmonyOS PC）或 OpenHarmony 设备
- **最低配置**：4 核 CPU、8GB 内存、20GB 可用磁盘空间
- **推荐配置**：8 核以上 CPU、16GB+ 内存、50GB+ SSD
- **必须开启**：“设置 --> 隐私和安全 -->运行来自非应用市场的扩展程序”

2.2 软件前提

组件	用途	获取方式
DevBox	提供 clang/lld 工具链	华为应用市场
GitNext	Git 版本控制	华为应用市场
DevNode	Node.js 运行时	华为应用市场
Python 3	框架运行环境	通过 cmd-pkgs 安装脚本自动安装
OpenDesk CLI	AI Agent 运行时	`npm install -g @bitclub.ai/opendesk-cli` ( )
AtomGit 账号	代码仓库访问与 PR 协作	atomgit.com

2.3 鸿蒙终端特有约束

- `/tmp` 目录只读 → 框架使用 `$TMPDIR` 指向 `$HOME/tmp`
- HMDFS 文件系统不支持 `ln -sf` 覆盖 → 使用 `rm -f && ln -s` 策略
- 所有可执行 ELF 必须签名 → 框架内置 `sign-elf` 子命令
-
- SELinux 限制 `chmod` → 记录为平台限制，不阻塞编译
- 无 `gcc` 仅有 `clang` → 环境变量强制 `CC=clang`、`CXX=clang++`
- 内核裁剪（`setreuid`/`chroot` 等未实现）→ 加入 EXCLUDE_LIST 排除

3. 安装部署 — 14 步设置向导详解

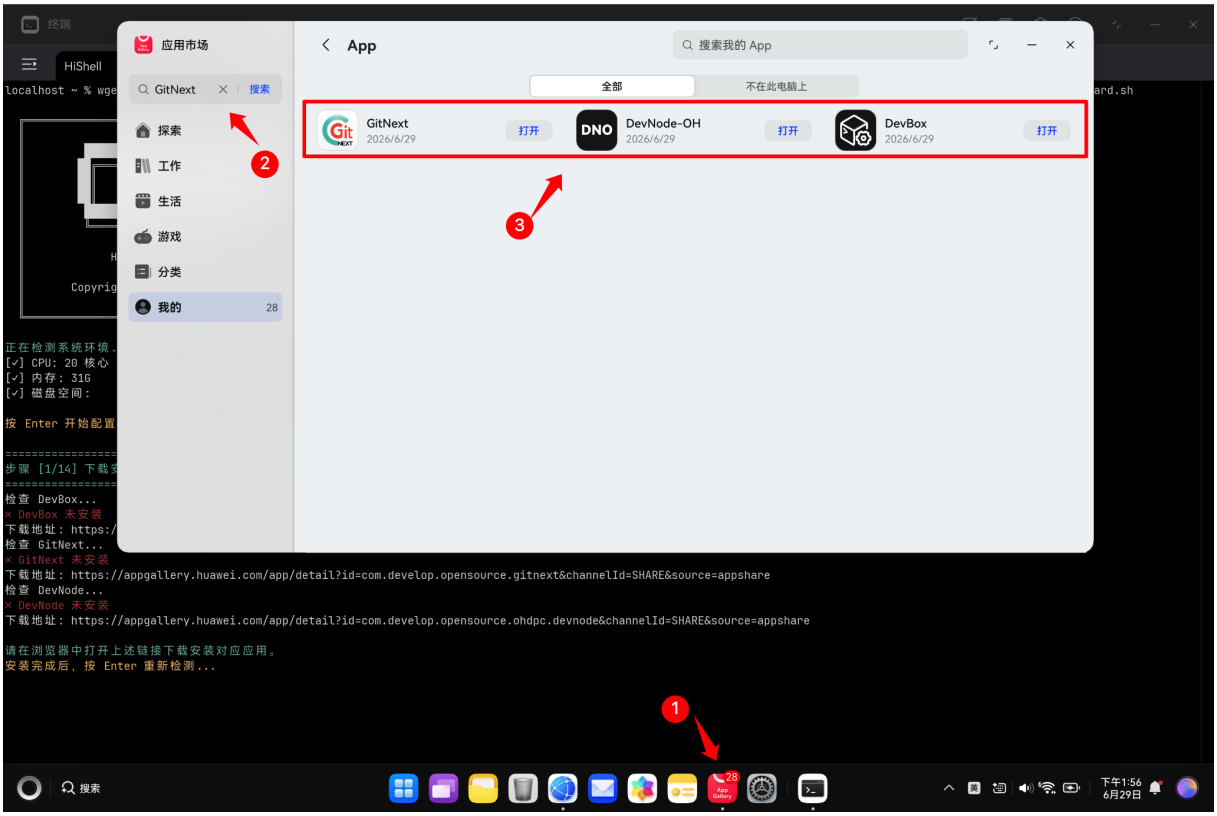
第 1 步：一键启动向导

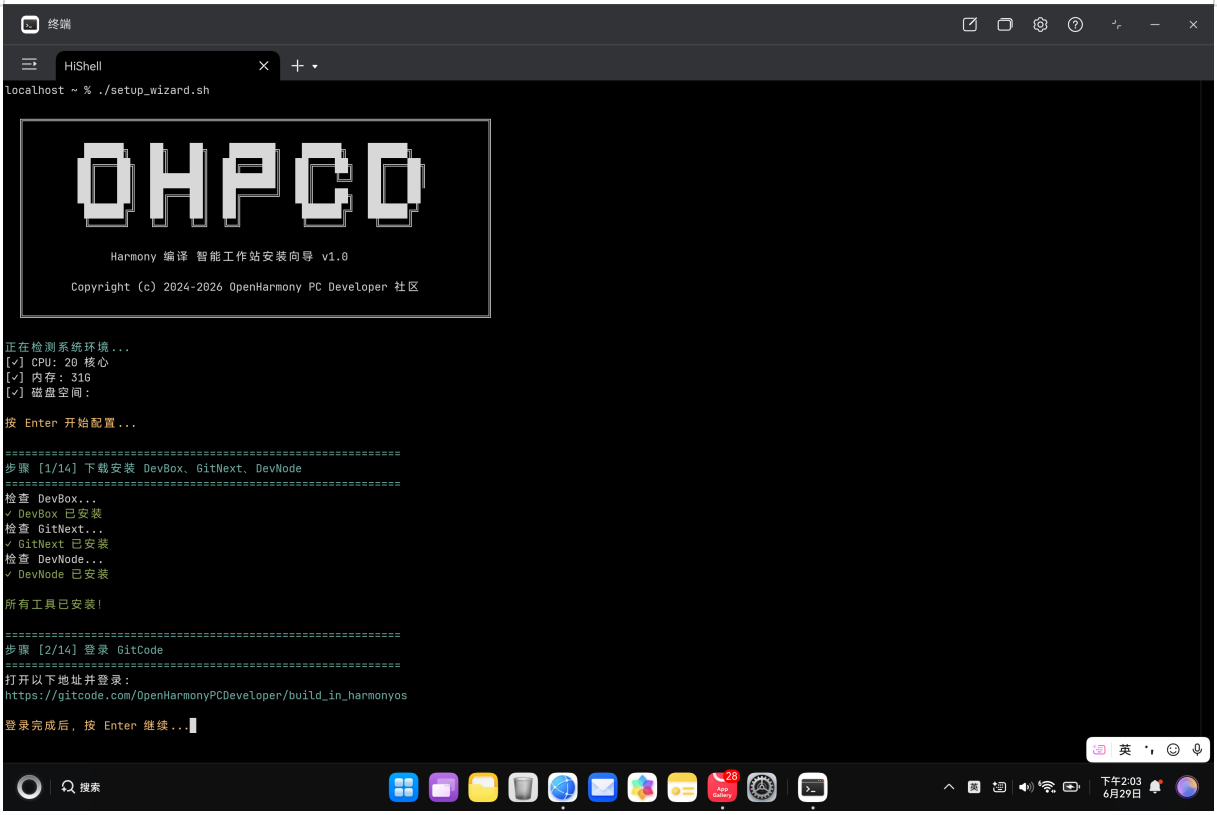
在鸿蒙PC上打开终端工具，输入以下命令开启安装向导。

r向导会显示系统环境检测结果（CPU 核心数、内存、磁盘空间），按 Enter 开始配置。

第 2 步：安装 DevBox、GitNext、DevNode

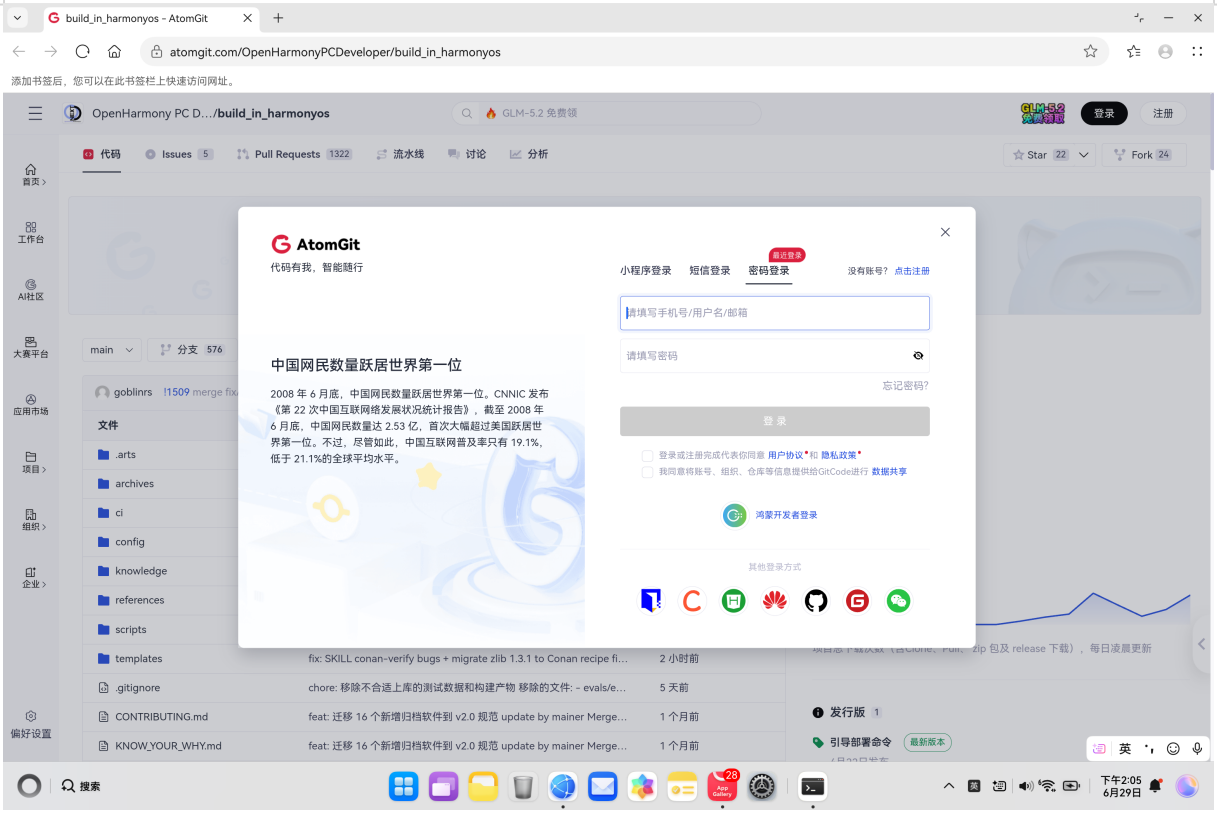
向导自动检测三款工具是否已安装。若未安装，会显示华为应用市场下载链接。**三款工具安装完成后，重启终端执行./setup_wizard.sh命令重启向导工具。**





第 3 步：登录 AtomGit

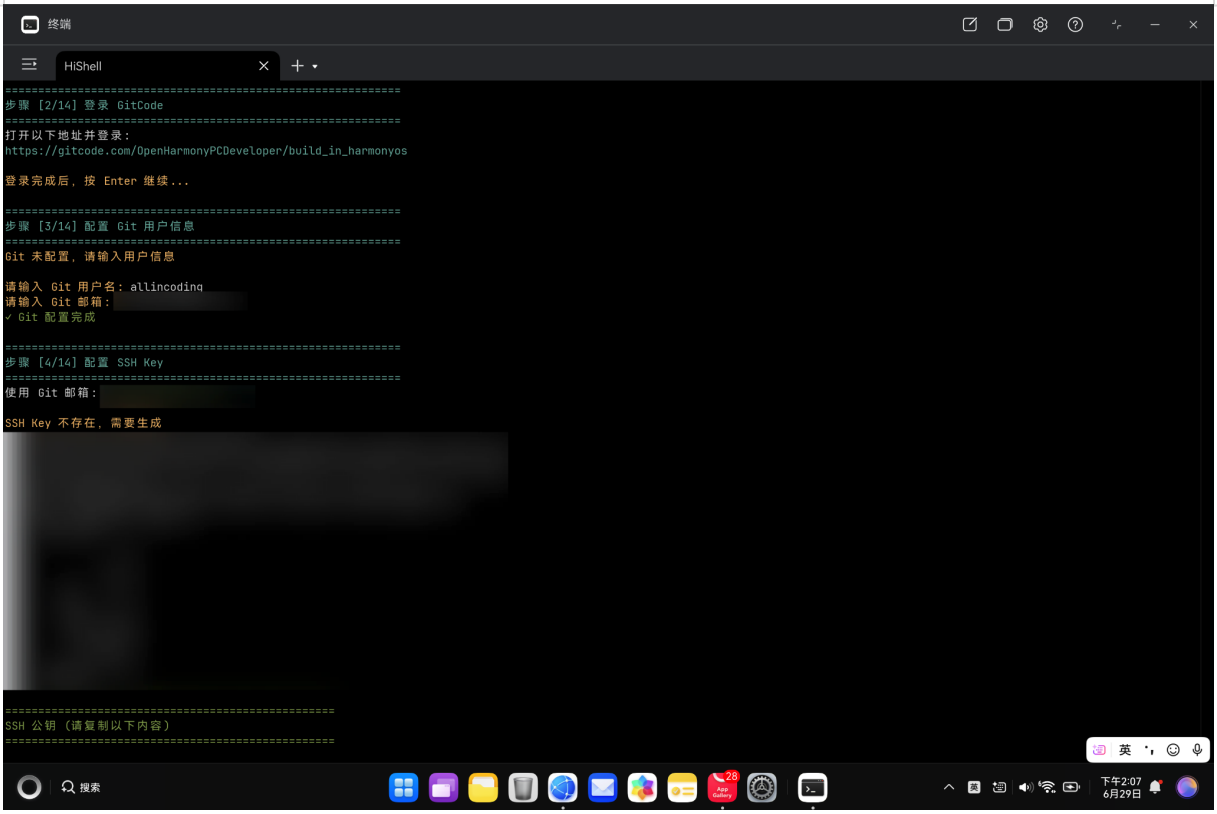
在浏览器中打开 build_in_harmonyos 仓库页面并登录 AtomGit 账号。**此步不可跳过。**



第 4 步：配置 Git 用户信息

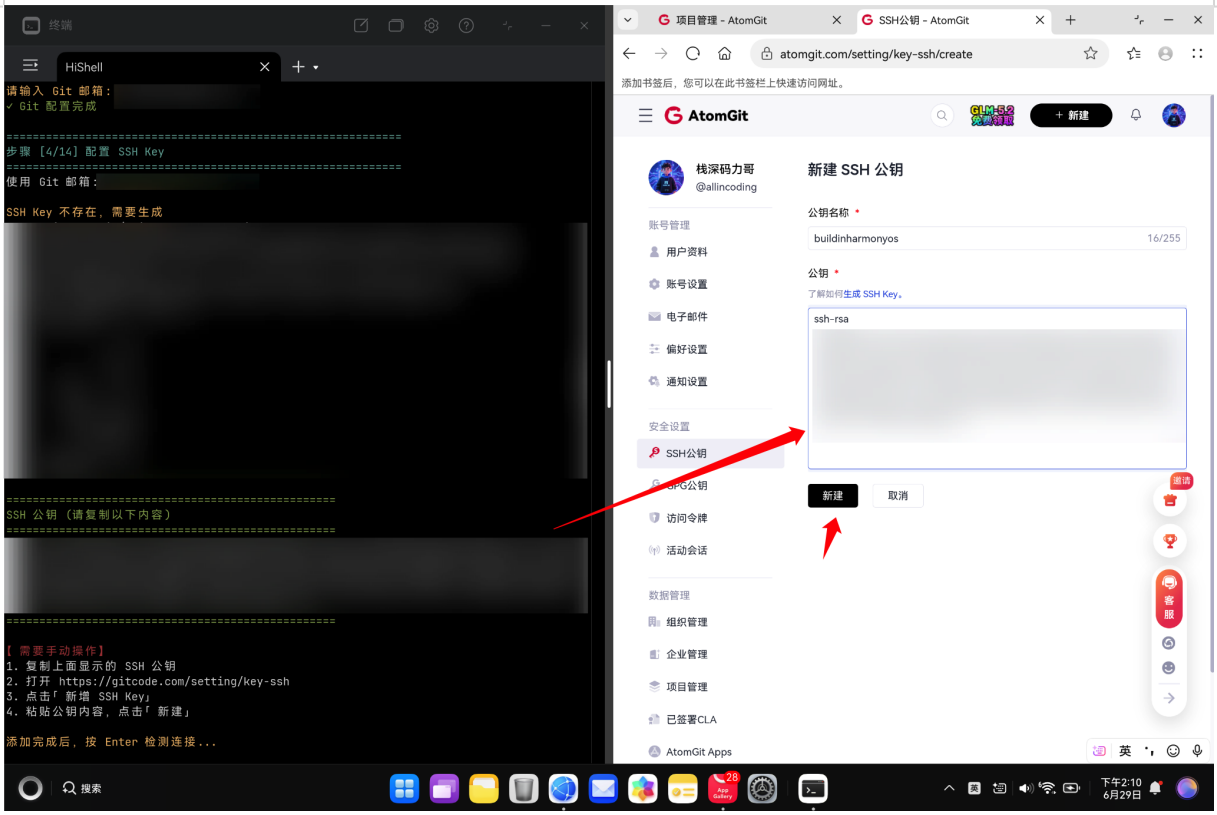
```
git config --global user.name "你的名字"
```

```
git config --global user.email "你的邮箱"
```

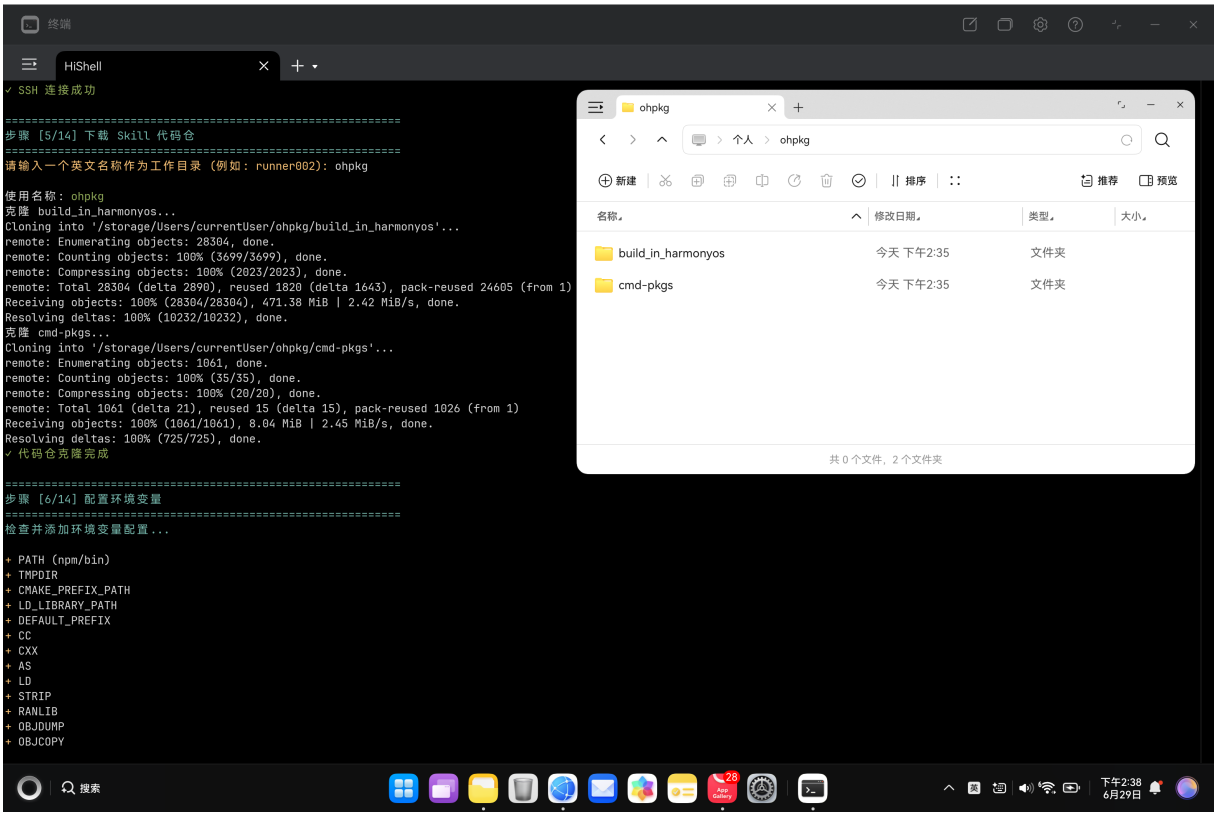


第 5 步：配置 SSH Key

- 若 `~/.ssh/id_rsa.pub` 已存在且连接成功 → 自动跳过
- 若不存在 → 自动执行 `ssh-keygen -t rsa -b 2048`，显示公钥引导手动添加到 🔗 AtomGit SSH 设置页。



第 6 步：下载 Skill 代码仓



最终目录结构：

/storage/Users/currentUser/${AGENT_NAME}/

├── build_in_harmonyos/ ←编译框架

└── cmd-pkgs/ ←预编译包分发

第 7 步：配置环境变量

向 ~/.zshrc 追加（自动备份原文件）：

路径类：

```
export PATH=/storage/Users/currentUser/npm/bin:/storage/Users/currentUser/usr/local/bin:$PATH

export TMPDIR=$HOME/tmp

export CMAKE_PREFIX_PATH=$HOME/usr/local

export LD_LIBRARY_PATH=$HOME/usr/local/lib:/data/service/hnp/magic-cli.org/magic-cli_1.0.0/lib:$LD_LIBRARY_PATH
```

```
export DEFAULT_PREFIX=$HOME/usr/local
```

编译工具链：

```
export CC="clang"

export CXX="clang++"

export LD="ld.lld"

export STRIP="llvm-strip"

export RANLIB="llvm-ranlib"

export AR="llvm-ar"

export NM="llvm-nm"

export OBJDUMP="llvm-objdump"

export OBJCOPY="llvm-objcopy"
```

```
export AS="llvm-as"
```

编译标志：

export CFLAGS="-fPIC -O2"

export CXXFLAGS="-fPIC -O2"

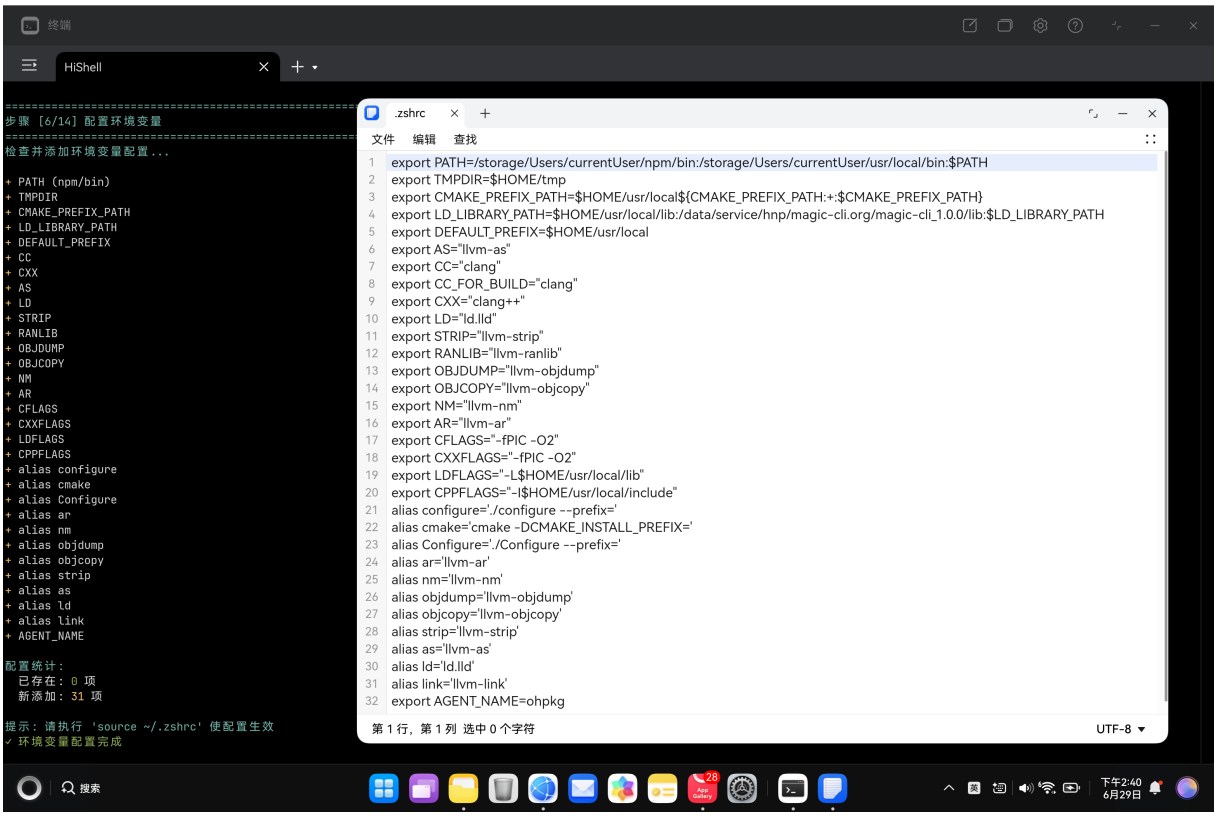
export LDFLAGS="-L$HOME/usr/local/lib"

export CPPFLAGS="-I$HOME/usr/local/include"便捷别名：

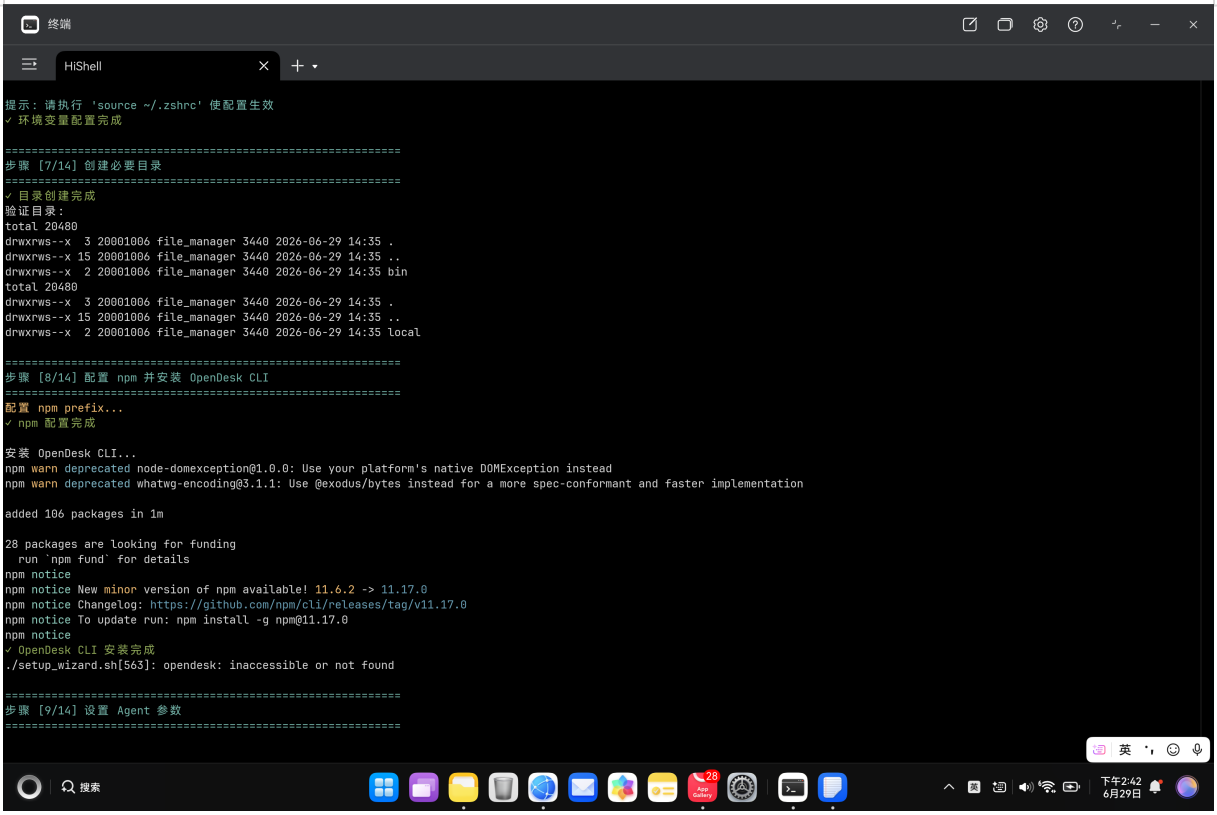
alias configure='./configure --prefix=$DEFAULT_PREFIX'

alias cmake='cmake -DCMAKE_INSTALL_PREFIX=$DEFAULT_PREFIX'

alias Configure='./Configure --prefix=$DEFAULT_PREFIX'执行 source ~/.zshrc 使其生效。



第 8 步：创建必要目录和配置 npm 并安装 OpenDesk CLI

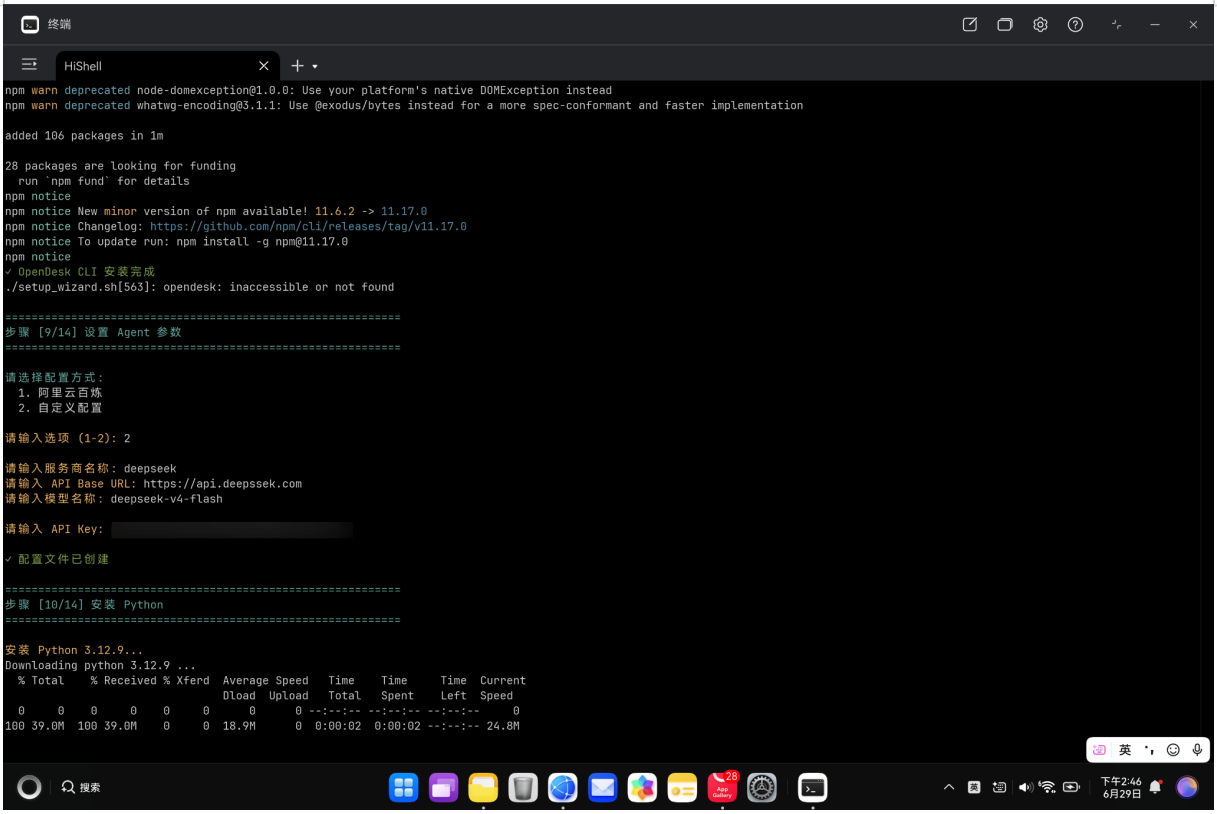


验证：`opendesk --version`（ 🔗 OpenDesk CLI 使用手册 ）

第 9 步：设置 Agent 参数（AI 模型配置）

- **选项 1：阿里云百炼**（推荐）— provider: `aliyun-cn`，默认模型 `qwen-plus`
- **选项 2：自定义配置** — 支持任意 OpenAI 兼容 API

需要输入 API Key。



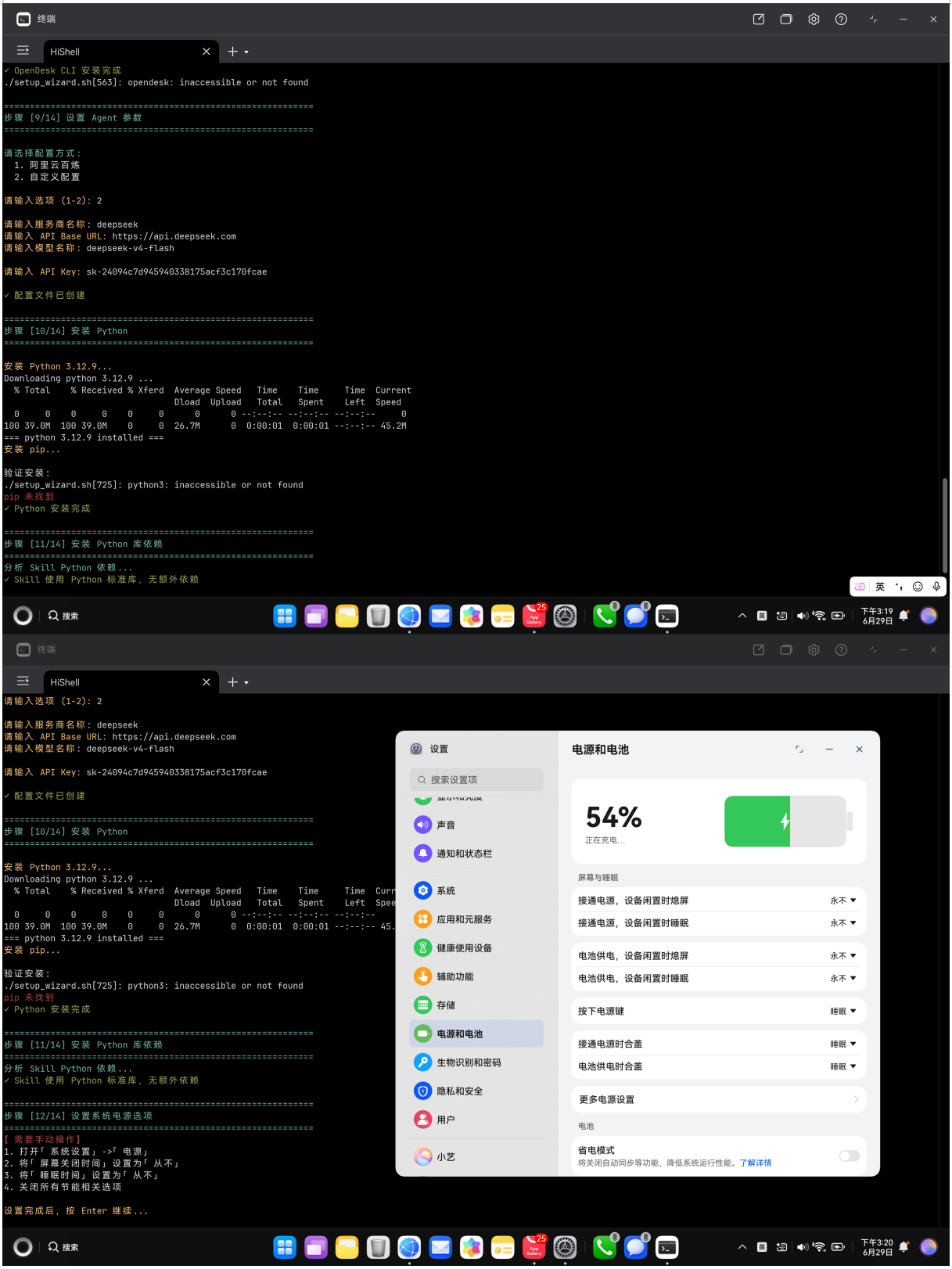
第 10 步：安装 Python和 Python 库依赖

框架核心设计为**零外部依赖**（纯 Python 标准库），requirements.txt 仅包含开发辅助工具。

第 11 步：设置系统电源选项（手动操作）

1. 打开「系统设置」→「电源和电池」
2. 将「屏幕关闭时间」和「睡眠时间」设为「从不」
3. 关闭所有节能相关选项

防止长时间编译中系统休眠导致中断。



第 13 步：注册 Skill 到 OpenDesk

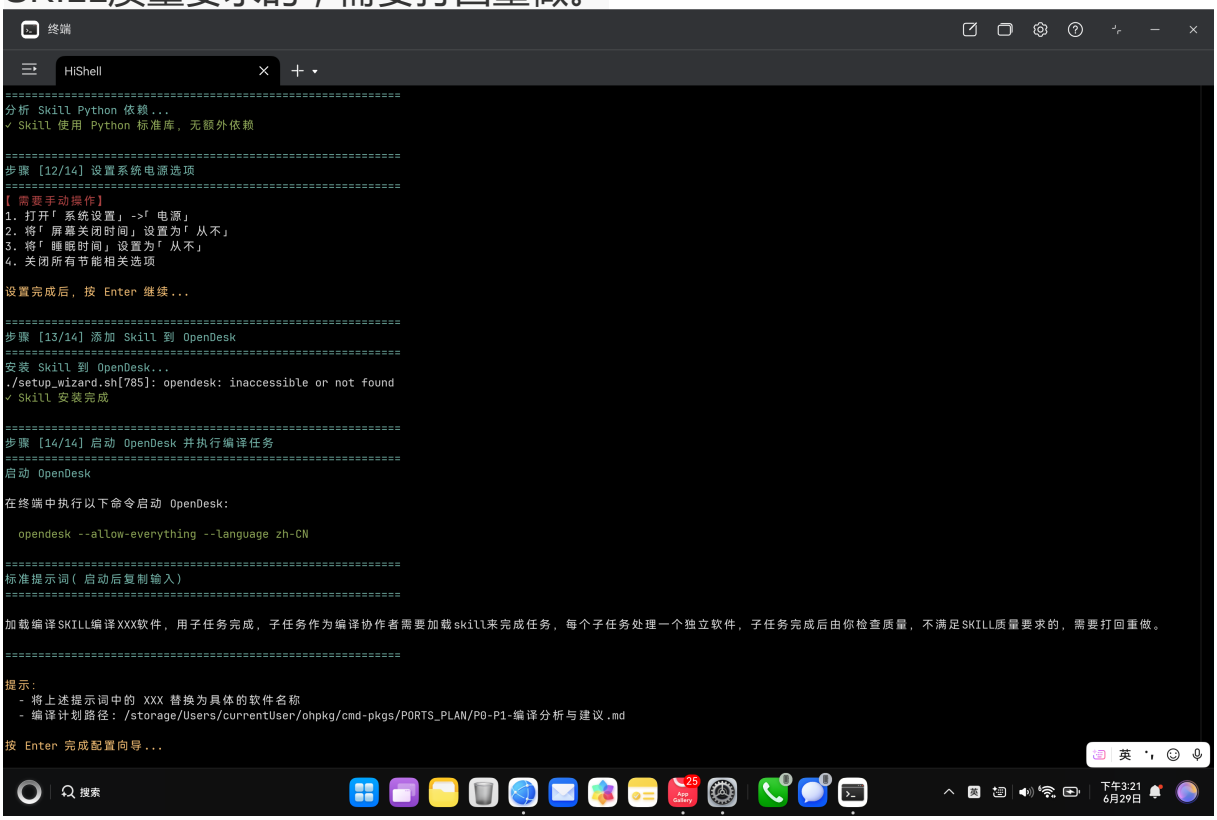
```
opendesk skill install
```

/storage/Users/currentUser/${AGENT_NAME}/build_in_harmonyos完成：启动

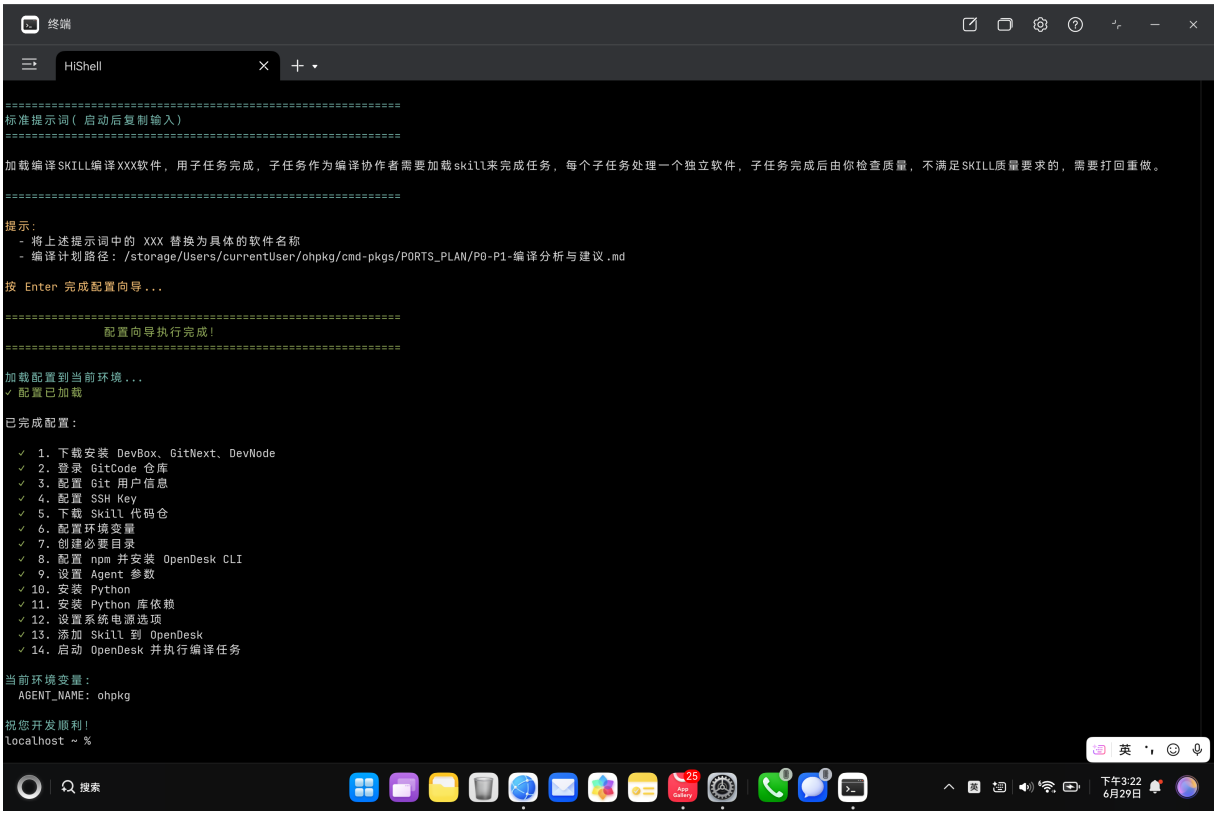
opendesk标准提示词：

加载编译SKILL编译XXX软件，用子任务完成，子任务作为编译协作者需要加载skill来完成任务，

每个子任务处理一个独立软件，子任务完成后由你检查质量，不满足SKILL质量要求的，需要打回重做。



环境配置完成！！！



3.1 HarmonyOS JavaScript/Node.js 三方库适配环境搭建

- DevBox（Clang 编译器）— 应用市场下载
- GitNext（Git 客户端） — 应用市场下载
- DevNode-OH — 应用市场下载
- HiShell（系统内置）

第一步 配置 Node.js 运行环境

1.1 创建 workspace 目录并复制 Node.js（终端输入）

mkdir -p ~/workspace/
cp -r /data/service/hnp/node.org ~/workspace/

如果 cp 报 "Operation not permitted"，用 tar 管道：
cd /data/service/hnp/node.org && tar cf - . | (cd ~/workspace && tar xf -)

1.2 对 Node.js 的 ELF 二进制文件签名

NODE_DIR="$HOME/workspace/node.org/node_v24.13.0"
for file in "$NODE_DIR/bin/node" "$NODE_DIR/bin/npm" "$NODE_DIR/bin/npx"; do

```
  if [ -f "$file" ]; then
    file "$file" 2>/dev/null | grep -q "ELF" && {
      llvm-objcopy --remove-section .codesign "$file" 2>/dev/null
      binary-sign-tool sign -selfSign 1 -inFile "$file" -outFile "$file" 2>/dev/null
      chmod +x "$file"
    }
  fi
done

find "$NODE_DIR" -type f \( -name "*.so*" -o -name "*.node" \) | while read file; do
  if file "$file" 2>/dev/null | grep -q "ELF"; then
    llvm-objcopy --remove-section .codesign "$file" 2>/dev/null
    binary-sign-tool sign -selfSign 1 -inFile "$file" -outFile "$file" 2>/dev/null
    chmod +x "$file"
  fi
done
```

1.3 修复 npm（鸿蒙上 /bin/bash 不存在的问题）

```
BIN_DIR="$NODE_DIR/bin"
printf '#!/usr/bin/env bash\n"%s" "%s" "$@"\n' \
  "$BIN_DIR/node" "$NODE_DIR/lib/node_modules/npm/bin/npm-cli.js" > "$BIN_DIR/npm"
chmod +x "$BIN_DIR/npm"
printf '#!/usr/bin/env bash\n"%s" "%s" "$@"\n' \
  "$BIN_DIR/node" "$NODE_DIR/lib/node_modules/npm/bin/npx-cli.js" > "$BIN_DIR/npx"
chmod +x "$BIN_DIR/npx"
```

1.4 添加 PATH 到 shell 配置

```
echo 'export PATH=$HOME/workspace/node.org/node_v24.13.0/bin:$PATH' >> ~/.zshrc
source ~/.zshrc
```

1.5 验证

```
node --version   # 应输出 v24.13.0
npm --version    # 应输出 11.x.x
```

第二步 克隆 JS 适配仓库

```
cd ~
git clone
git@gitcode.com:OpenHarmonyPCDeveloper/JavaScript_Package_For_HarmonyOS.git
```

如果提示 "already exists"，说明之前已克隆过：

```
cd ~/JavaScript_Package_For_HarmonyOS && git pull
```

【第三步】配置编译环境变量

```
export DEFAULT_PREFIX=$HOME/usr/local
export TMPDIR=$HOME/tmp
export CC=clang
export CXX=clang++
export AR=llvm-ar
export CFLAGS="-fPIC -O2"
export LDFLAGS="-L$DEFAULT_PREFIX/lib"
export CPPFLAGS="-I$DEFAULT_PREFIX/include"
```

建议写入 ~/.zshrc 永久生效：

```
cat >> ~/.zshrc << 'EOF'

# JS/Python 原生编译环境
export DEFAULT_PREFIX=$HOME/usr/local
export TMPDIR=$HOME/tmp
export CC=clang
export CXX=clang++
export AR=llvm-ar
export CFLAGS="-fPIC -O2"
export LDFLAGS="-L$DEFAULT_PREFIX/lib"
export CPPFLAGS="-I$DEFAULT_PREFIX/include"
EOF

source ~/.zshrc
```

==========

【第四步】安装 7 个 JS Skill

```
cd ~/JavaScript_Package_For_HarmonyOS

opendesk skill install ./skill/china-mirror-resolver
opendesk skill install ./skill/harmonyos-nodejs-analyze
opendesk skill install ./skill/harmonyos-nodejs-batch-validate
opendesk skill install ./skill/harmonyos-nodejs-migrate
opendesk skill install ./skill/harmonyos-nodejs-package
opendesk skill install ./skill/harmonyos-nodejs-test
opendesk skill install ./skill/harmonyos-nodejs-validate
```

注意路径是 ./skill/xxx（单数），不是 ./skills/xxx
共 7 个，harmonyos-nodejs-upload 不存在，跳过

==========

【第五步】启动 OpenDesk

opendesk --allow-everything --language zh-CN

==========

【第六步】输入提示词开始适配

启动后粘贴以下内容（将 XXX 替换为目标 npm 包名）：

加载harmonyos-nodejs-validate Skill验证迁移XXX npm包，
用子任务完成，每个子任务处理一个独立npm包，
子任务完成后由你检查质量

例如适配 openai-codex：

加载harmonyos-nodejs-validate Skill验证迁移openai-codex npm包，
用子任务完成，每个子任务处理一个独立npm包

3.2 HarmonyOS Python 三方库适配环境搭建 —— 正确步骤

【前置条件】

确保鸿蒙设备上已安装：
- DevBox（Clang 编译器）
- GitNext（Git 客户端）
- HiShell（系统内置）

==========

【第一步】克隆 Python 适配仓库

打开终端，执行：

cd ~
git clone
git@gitcode.com:OpenHarmonyPCDeveloper/Python_Package_For_HarmonyOS.git

如果提示 "already exists"，说明之前已克隆过，

直接 cd Python_Package_For_HarmonyOS && git pull 即可。

==========

【第二步】配置环境变量

执行以下命令：

```
export DEFAULT_PREFIX=$HOME/usr/local
export TMPDIR=$HOME/tmp
export CC=clang
export CXX=clang++
export AR=llvm-ar
export CFLAGS="-fPIC -O2"
export LDFLAGS="-L$DEFAULT_PREFIX/lib"
export CPPFLAGS="-I$DEFAULT_PREFIX/include"
```

如果想永久生效（推荐），追加到 shell 配置：

```
cat >> ~/.zshrc << 'EOF'

# Python 原生编译环境
export DEFAULT_PREFIX=$HOME/usr/local
export TMPDIR=$HOME/tmp
export CC=clang
export CXX=clang++
export AR=llvm-ar
export CFLAGS="-fPIC -O2"
export LDFLAGS="-L$DEFAULT_PREFIX/lib"
export CPPFLAGS="-I$DEFAULT_PREFIX/include"
EOF

source ~/.zshrc
```

==========

【第三步】检查环境是否就绪

```
bash ~/Python_Package_For_HarmonyOS/scripts/check_environment.sh
```

期望结果：全部　　通过，0 项　　失败
Rust 相关的　　警告可忽略（只有部分库需要）。

==========

【第四步】安装三个 Python Skill

cd ~/Python_Package_For_HarmonyOS
opendesk skill install ./skills/harmonyos-python-native-build
opendesk skill install ./skills/harmonyos-python-native-troubleshoot
opendesk skill install ./skills/source-prepare

注意：一定要在仓库目录内用 ./skills/... 的相对路径，
不要在目录外写 Python_Package_For_HarmonyOS/skills/...，
OpenDesk 无法识别。

==========

【第五步】启动 OpenDesk

opendesk --allow-everything --language zh-CN

==========

【第六步】输入提示词开始适配

启动后复制以下提示词粘贴进去（将 XXX 替换为目标库名）：

加载harmonyos-python-native-build Skill适配XXX Python库，
用子任务完成，每个子任务处理一个独立Python库，
子任务完成后由你检查质量

例如适配 numpy：

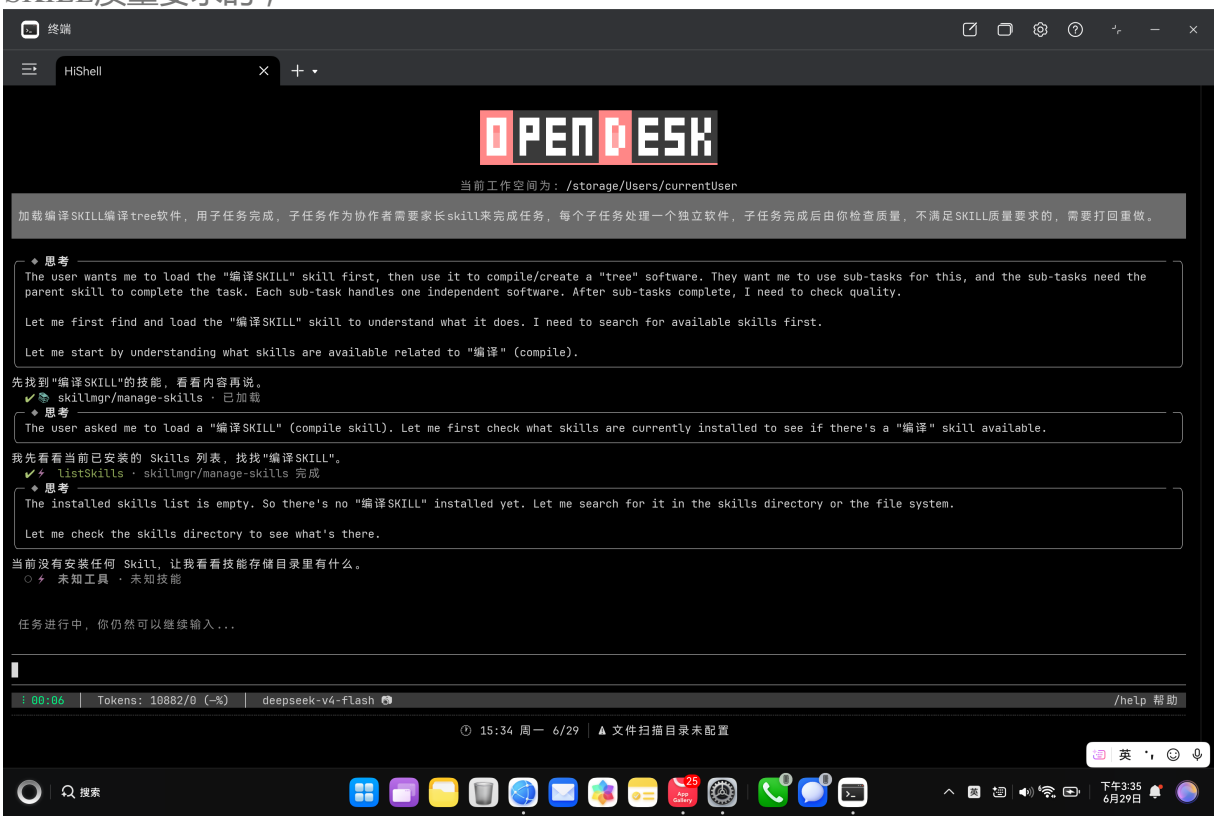
加载harmonyos-python-native-build Skill适配numpy Python库，
用子任务完成，每个子任务处理一个独立Python库，
子任务完成后由你检查质量

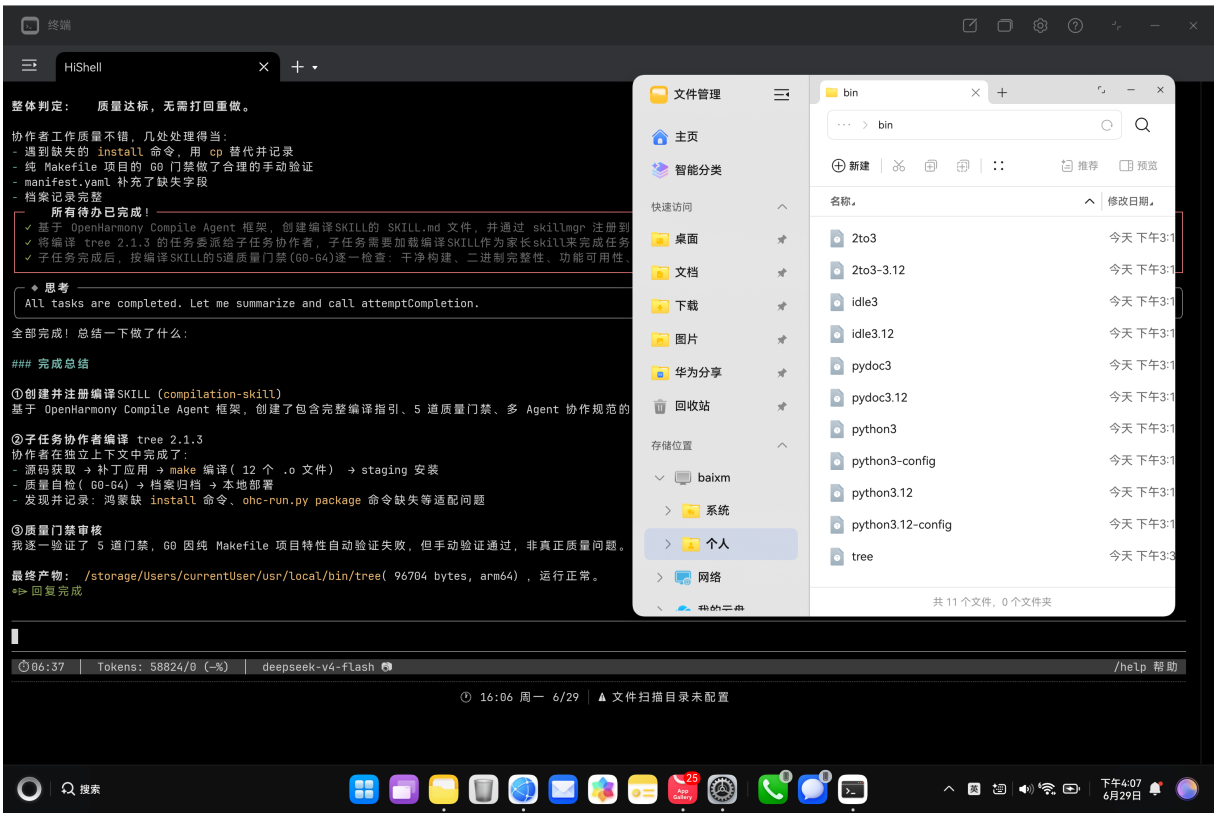
4. 通过 AI Agent完成贡献（推荐）

在 OpenDesk 中发送库适配自然语言提示词：

"加载编译SKILL编译tree软件，用子任务完成，子任务作为编译协作者需要加载skill来完成任务，每个子任务处理一个独立软件，子任务完成后由你检查质量，不满足

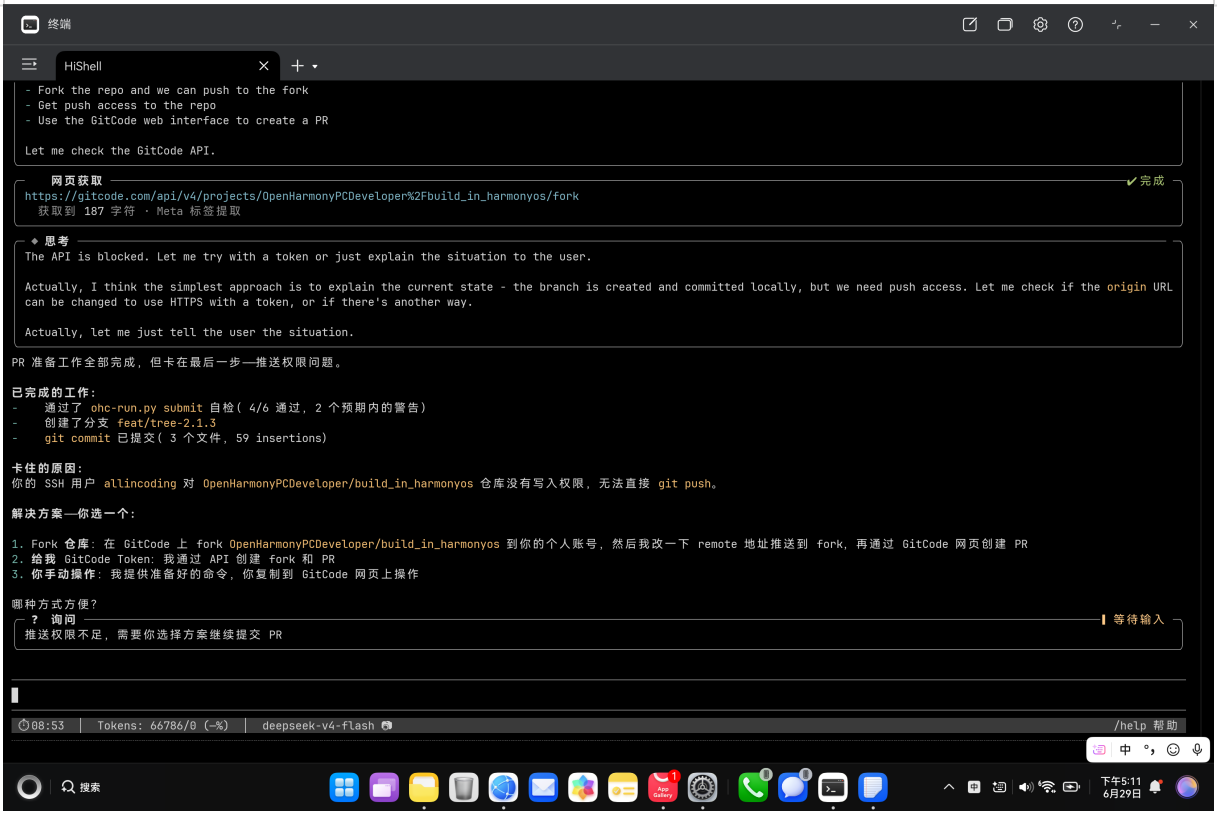
SKILL质量要求的，





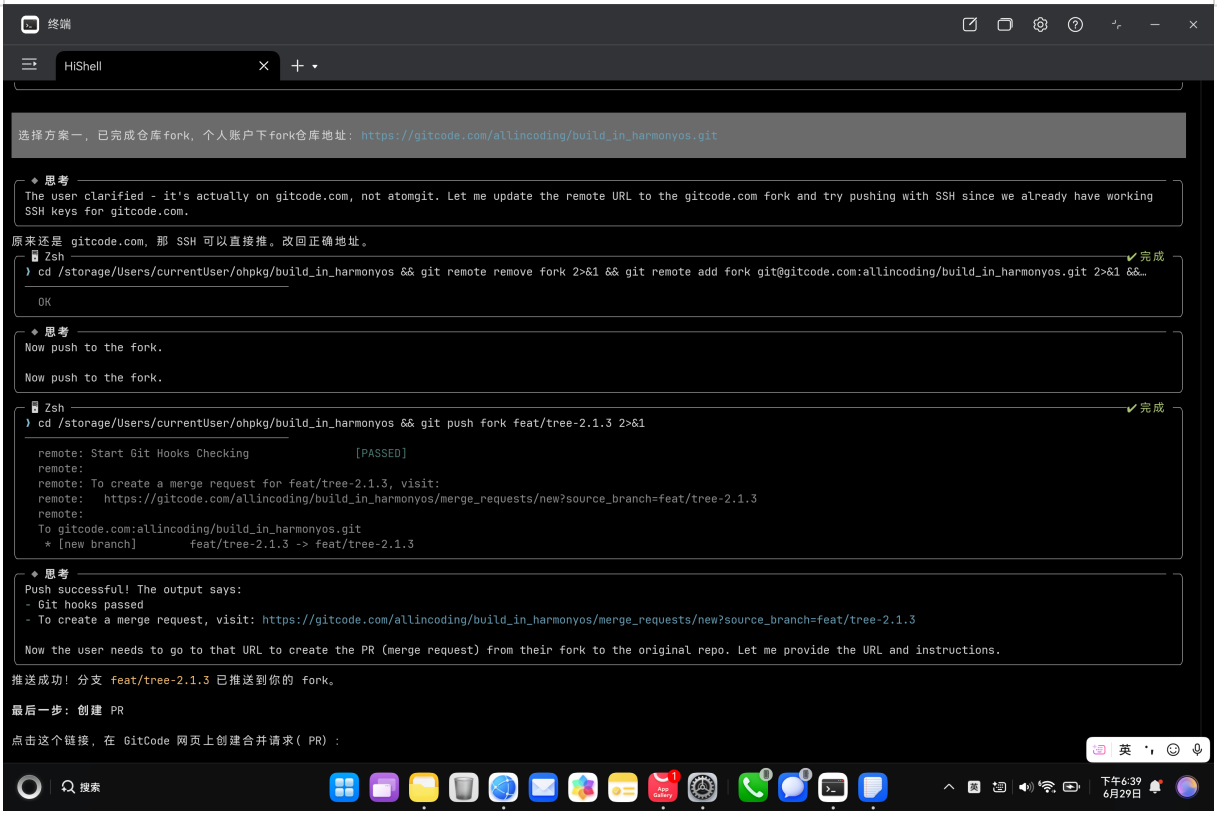
在 OpenDesk 中发送提PR自然语言提示词：

已完成质量门禁审核，现在需要以PR方式提交tree。

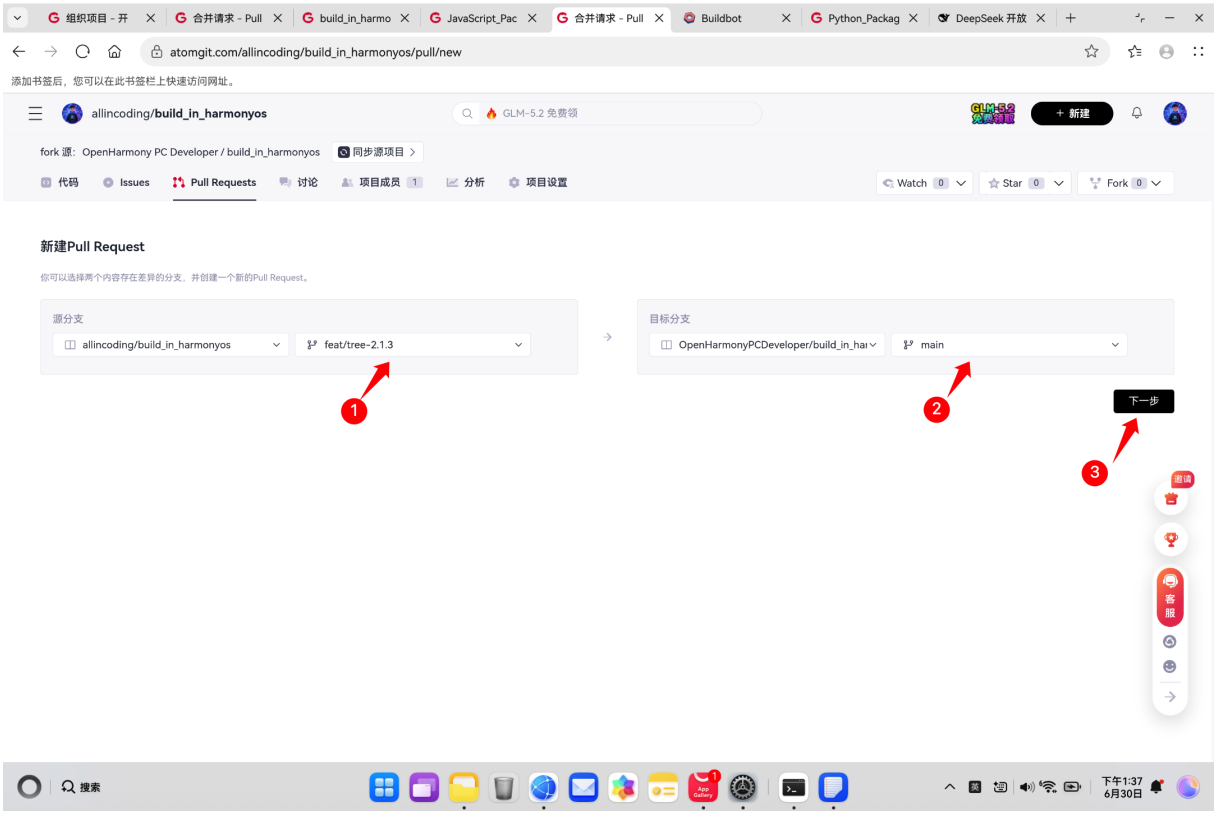


从结果可以看出来，是因为权限的问题，导致没有推送成功。这个原因是build-in-harmonyos仓库需要成为项目开发者才具有一键提PR权限。我们可以选择Fork仓库，将build-in-harmonyos工程fork到个人账号下，然后**在 OpenDesk 中发送Fork仓库地址自然语言提示词：**

选择方案一，已完成仓库fork，个人账号下fork仓库地址：
https://gitcode.com/allincoding/build_in_harmonyos.git



上述方式还需要你手动在fork到个人仓库下的build-in-harmonyos仓库中查看是否有提交的分支信息feat/tree-2.1.3，然后再提交合入PR请求。



推荐方式，在build-in-harmonyos仓库，点击仓库邀请链接，并通过issue告诉仓库

管理员需要加入当前项目，这种方式能够真正达到无感知提交PR。



python的贡献指南同上；

js的最新的贡献指南做了小的调整：

1、使用上游地址先fork到atomgit自己的仓库中，上游地址：
GitHub - ohos-ports/ohos-ports · GitHub
然后可以遵循以上的cpp和python的使用agent来贡献pr的方法来直接做适配代码

2、个人仓验证完成后，提交PR到github上游仓库：ohos-ports/ohos-ports
3、Committer合入PR后触发流水线构建，构建成功自动发布到npmjs.com官网

5. 以cunit命令行工具为例梳理整个过程（重点）

前面是环境验证，通过cunit命令行适配流程熟练掌握使用OpenHarmony Compile Agent工具参与贡献流程。

第一步：查看 任务清单 ，在 build-in-harmonyos 仓库Issues页签按照以下格式提交认领Issue。提交成功后可在Issues页签查看到当前的认领记录。**同时，还需要联系赛事小助手，同步更新认领清单，防止未及时更新被其他人认领。**

```
# Issue标题

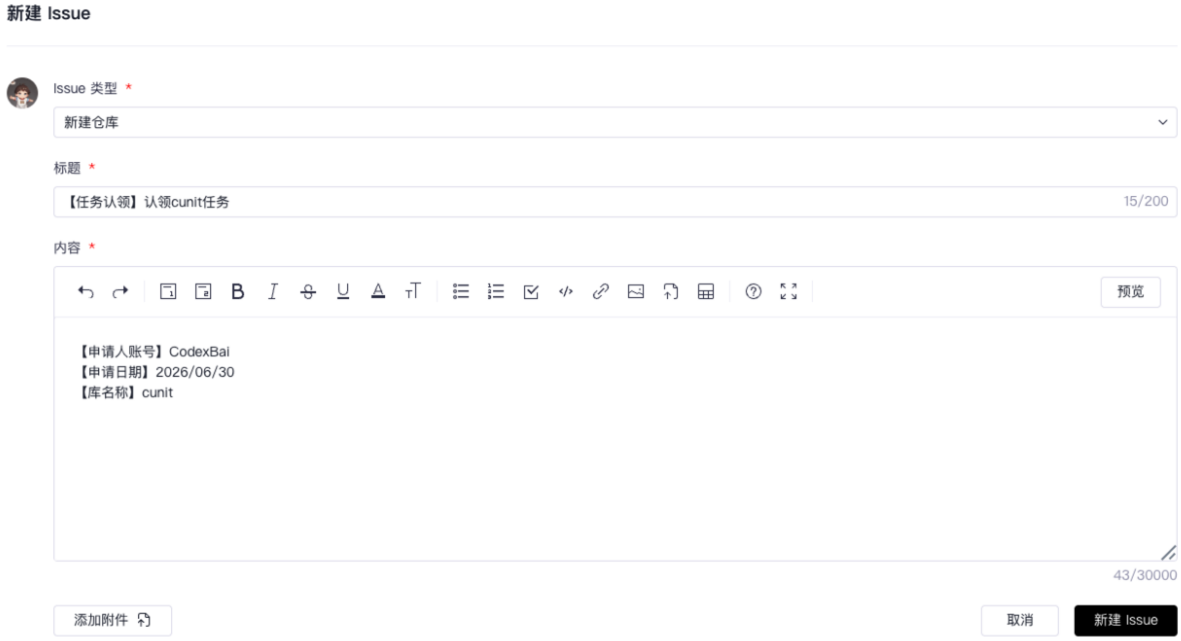
【任务认领】认领{库名称}任务


# Issue正文


【申请人账号】{填写申请人atomgit用户名}
```

【申请日期】{填写申请日期，如：2026/06/30}

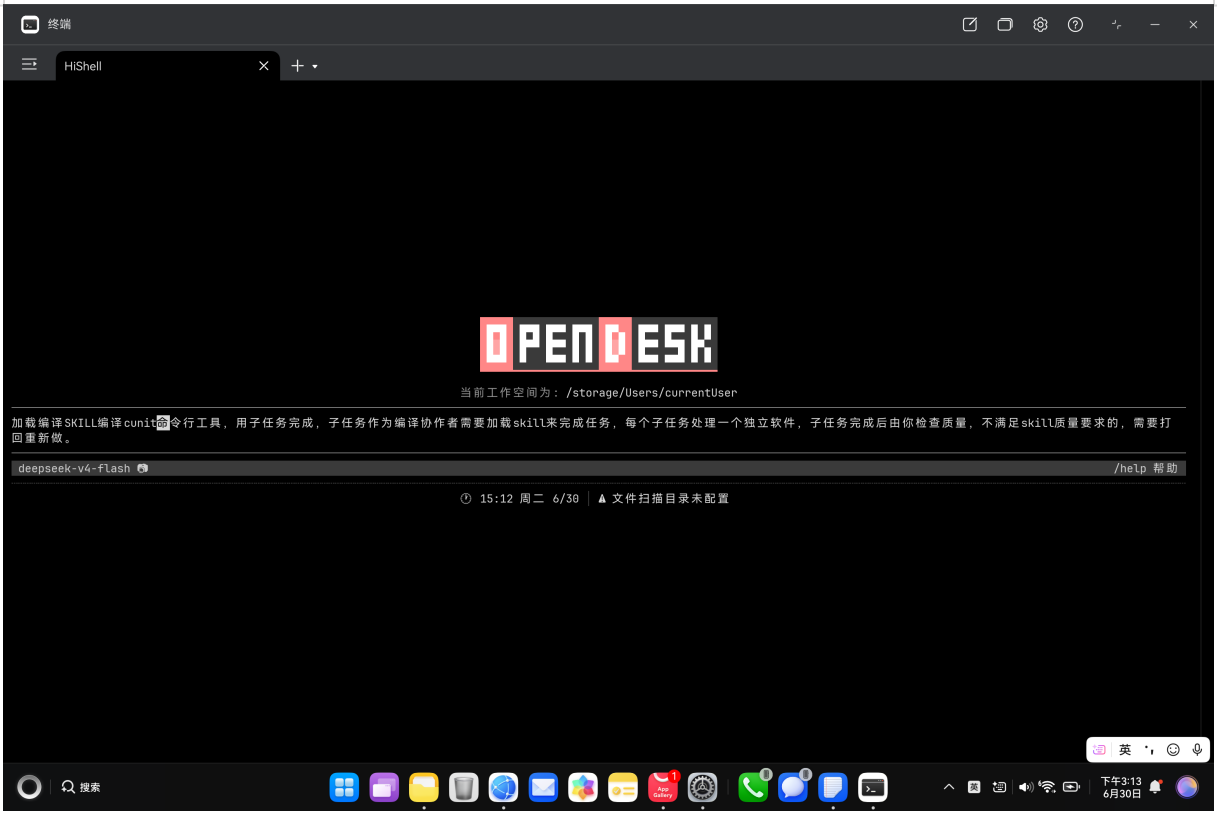
【库名称】{填写任务库名称}



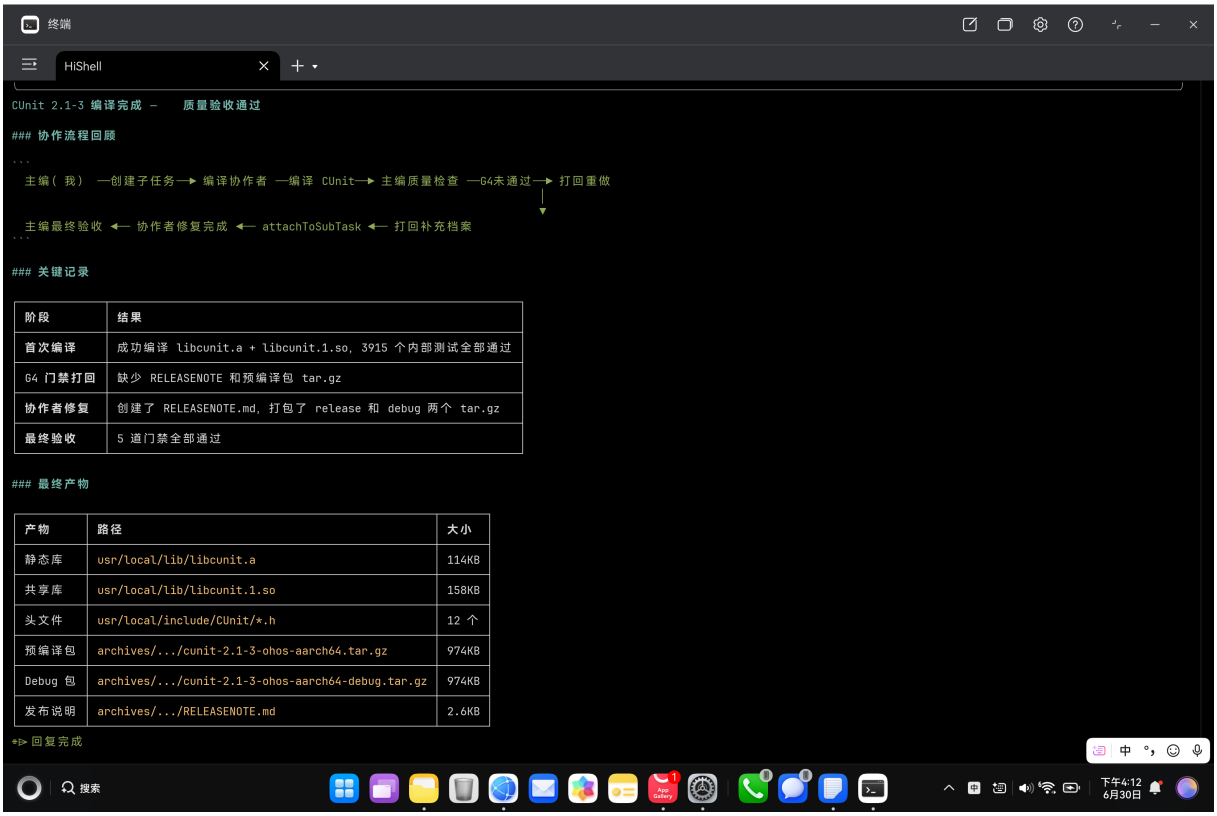


第二步：在鸿蒙PC上打开终端，输入opendesk命令启动OpenDesk（需要确保前面的环境安装没有问题），输入开始适配任务库的提示词：

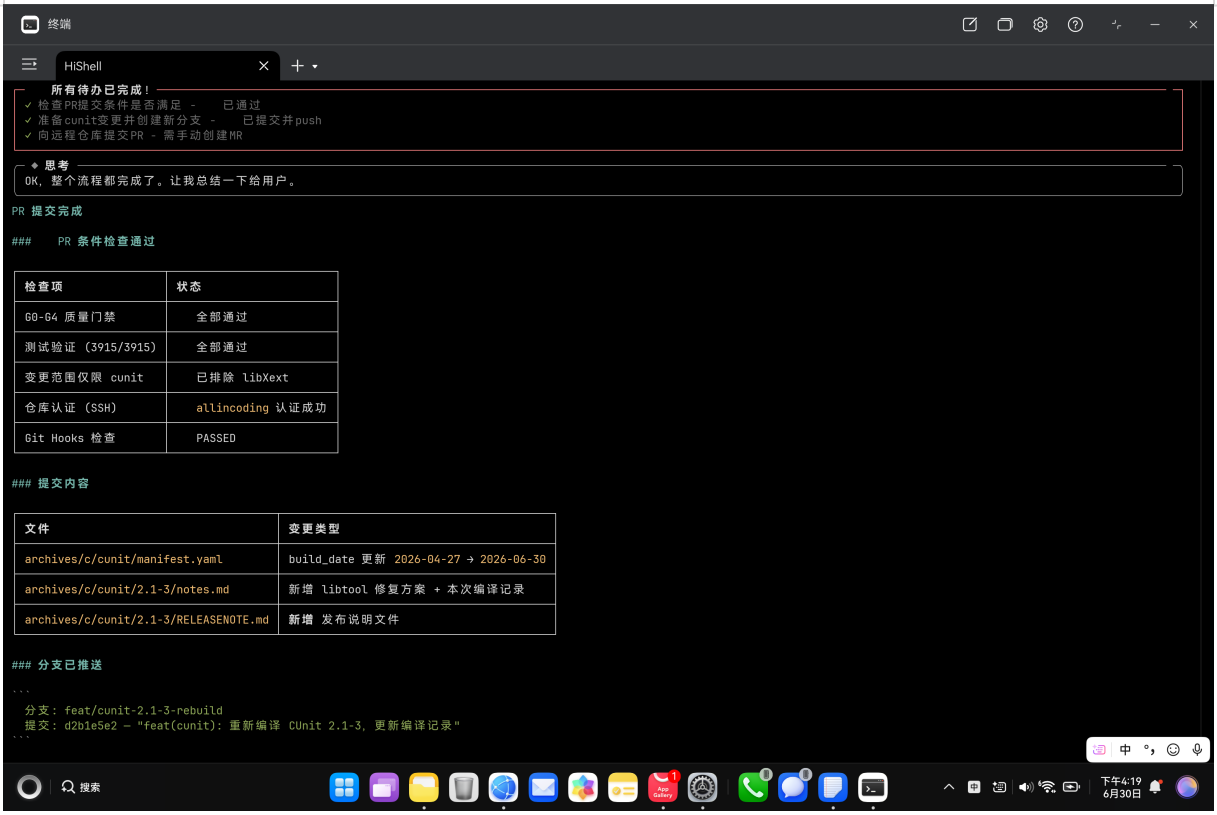
"加载编译SKILL编译cunit命令行工具，用子任务完成，子任务作为编译协作者需要加载skill来完成任务，每个子任务处理一个独立软件，子任务完成后由你检查质量，不满足SKILL质量要求的，需要打回重做。"



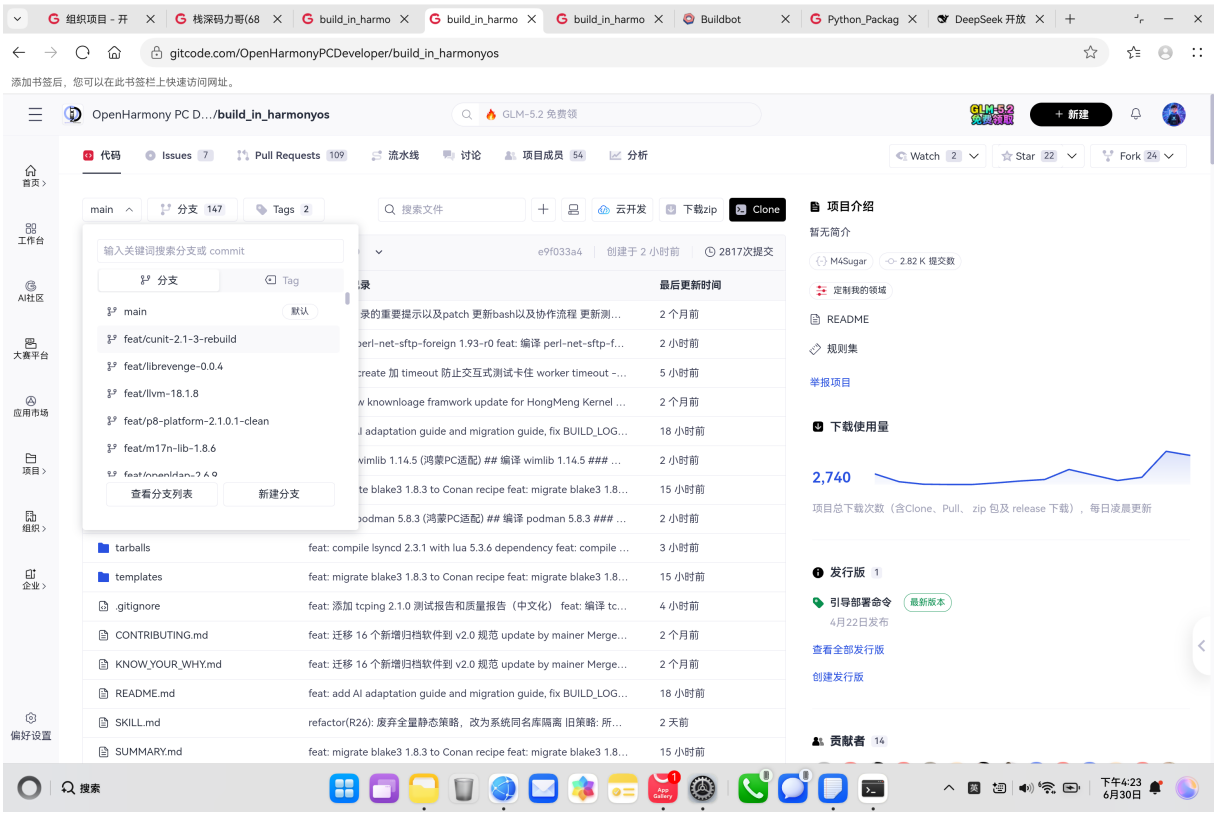
这是一个耗时的过程，等待任务完成后，会显示编译总结。为什么耗时呢，它会自己解决编译过程中出现的各类问题，直到编译成功。



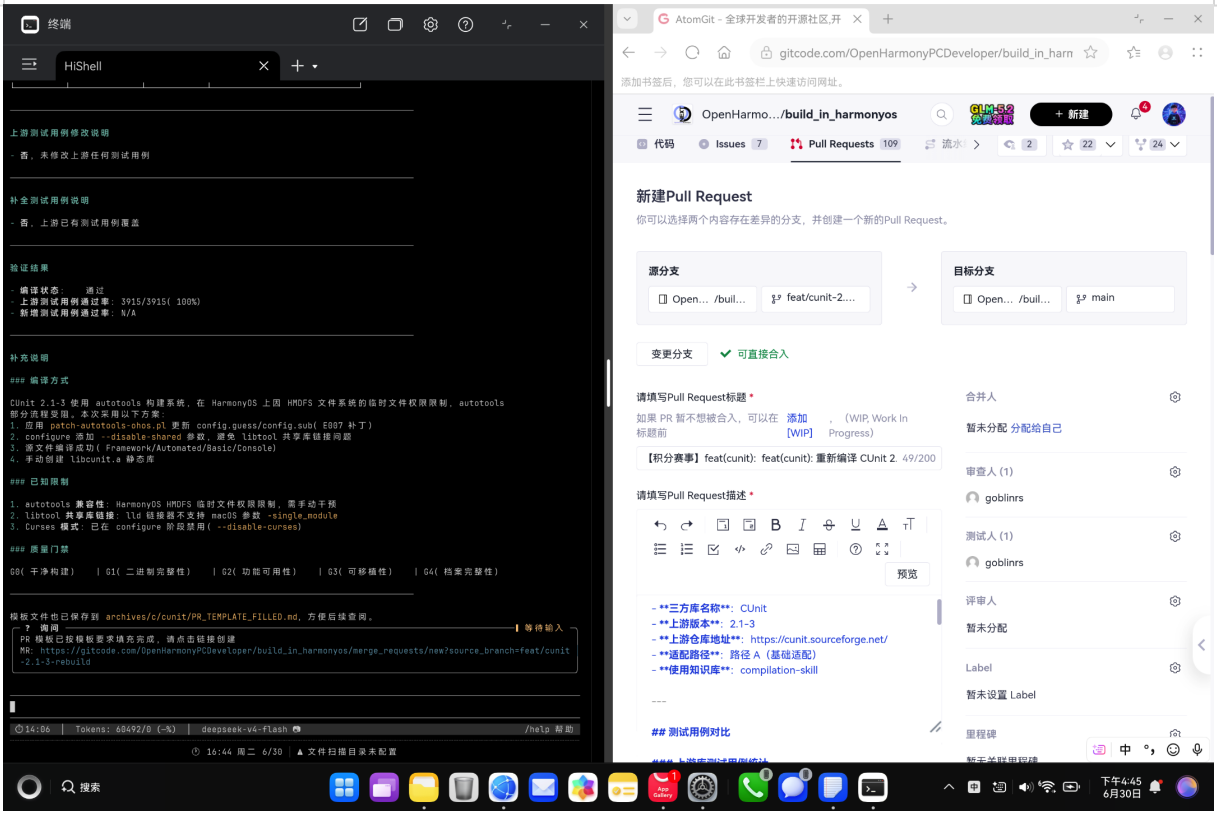
第三步：输入提交PR提示词：已完成编译，检查是否满足提交PR，若满足，以PR方式提交cunit。



第四步：在build-in-harmonyos 仓库查看提交的分支是否存在。



第五步： 拷贝PR链接在浏览器中打开，PR标题前需要增加【积分赛事】标签，并将生成的PR信息拷贝的PR正文。



第六步：等待PR合入，PR合入后，当前任务结束。